

```

# HumAL

HumAIne Active Learning Platform - An integrated system for human-in-the-
loop machine learning workflows, featuring automated ticket
classification, resolution generation, and active learning capabilities.

## Architecture
![Architecture Diagram] (docs/images/smart_ticketing_architecture.svg)

## Features

- **Ticket Classification**: Automated team routing
- **Resolution Generation**: LLM-powered ticket resolution suggestions
  using RAG
- **Active Learning**: Interactive model training with human feedback
- **Explainable AI**: LIME-based model explanations
- **Modern UI**: React + TypeScript frontend with real-time updates

## Prerequisites

- **Python 3.8+** (for backend)
- **Node.js 18+** (for frontend)
- **uv** or **pip** (for Python package management)
- **npm** (for frontend package management)
- **CUDA Toolkit** (optional, for GPU acceleration - will be auto-
  detected)

## Quick Start (local)

### 1. Install uv (Optional but Recommended)

```bash
powershell -c "irm https://astral.sh/uv/install.ps1 | iex"
```

### 2. Create and Activate Virtual Environment

```bash
Create virtual environment with uv (recommended)
uv venv al_api_venv

Or with Python's built-in venv
python -m venv al_api_venv

Activate
al_api_venv\Scripts\activate # Windows
source al_api_venv/bin/activate # Unix / macOS
```

### 3. Run Automated Installation

```bash
Basic installation (auto-detects CUDA and package manager)

```

```
python install.py

Force CPU-only installation
python install.py --cpu-only

Force use of pip instead of uv
python install.py --use-pip
```
```

The installer will:

- ,úi Auto-detect CUDA and install PyTorch with GPU support (if available)
- ,úi Install all backend dependencies from requirements.txt
- ,úi Install all frontend dependencies
- ,úi Verify the installation and display GPU/CPU status

4. Prepare Data and Models

```
```bash
Create directories (if they don't exist)
mkdir backend\data
mkdir backend\models
```
```

Place the following in `backend/data/`:

- Resolution KB: `User\_Request\_last\_team\_ANON.csv`
- Active Learning datasets: `al\_demo\_train\_data.csv`,
`al\_demo\_test\_data.csv`, `al\_demo\_train\_labels\_dispatch.csv`

Place pre-trained models in `backend/models/`:

- `perfect\_team\_classifier/` folder
- `ticket\_classifier\_model/` folder

5. Configure Environment

```
```bash
Copy .env.example to .env (if .env.example exists)
copy .env.example .env # Windows
cp .env.example .env # Unix / macOS
```
```

Edit `.env` and add your OpenAI API key:

```
```
OPENAI_API_KEY=your-openai-api-key-here
```
```

6. Run the Application

```
```bash
.\start-dev.bat # Windows
./start-dev.sh # Unix / macOS
```
```

Access Points

After starting both services:

- ****Frontend Application****: <http://localhost:5173>
- ****Backend API****: <http://localhost:8000>
- ****API Documentation****: <http://localhost:8000/docs>

Quick Start (Docker)

1. Configure Environment

```
```bash
copy .env.example .env # Windows
cp .env.example .env # Unix / macOS
```
```

Edit `.env` and add your OpenAI API key:

```
```
OPENAI_API_KEY=your-openai-api-key-here
```
```

2. Run with Docker

```
```bash
docker-compose up
```
```

Access Points

The services will be available at:

- ****Frontend Application****: <http://localhost:5173>
- ****Backend API****: <http://localhost:8000>
- ****API Documentation****: <http://localhost:8000/docs>

Available Pages

- ****Home**** (`/`) - Landing page
- ****Training**** (`/training`) - Model training interface
- ****Dispatch Labeling**** (`/dispatch-labeling`) - Label dispatch data
- ****Ticket Resolution**** (`/ticket-resolution`) - Generate automated responses
- ****Inference**** (`/inference`) - Run model inference

Documentation

- ****[ARCHITECTURE.md] (docs/ARCHITECTURE.md)**** - System architecture, components, data flow, and ML pipeline details

```
- **[API.md](docs/API.md)** - REST API endpoint reference with  
request/response examples  
- **[USER_GUIDE.md](docs/USER_GUIDE.md)** - Step-by-step guide for using  
all platform features  
- **[DEVELOPMENT.md](docs/DEVELOPMENT.md)** - Development setup,  
workflows, and contribution guidelines  
- **[DEPLOYMENT.md](docs/DEPLOYMENT.md)** - Docker deployment and  
production setup instructions
```

Project Structure

...

```
HumAL/  
├── backend/  
│   ├── app/                # FastAPI application  
│   ├── data/               # CSV datasets  
│   ├── models/             # Saved models  
│   └── tests/              # Backend tests  
├── frontend/               # React + Vite frontend  
├── start-dev.bat           # Windows startup script  
├── start-dev.sh            # Unix/Linux startup script  
├── requirements.txt        # Backend dependencies  
├── install.py              # Automated dependency installer  
└── SETUP.md                # Detailed setup guide  
...
```

Acknowledgements

The project has received funding from the European Union's funded Project HEU HUMAINE under Grant Agreement No. 101120218.

License

Apache License 2.0